Assignment - 3

Section A

Ans 1 : Clocking methodology is a synchronization mechanism in processors where all operations are coordinated using a clock signal. Each clock pulse triggers data movement between registers and execution in functional units, ensuring orderly and timed execution of instructions.

Ans 2 : A clock cycle is the smallest unit of time in processor operation. Each instruction is broken into steps executed in one or more clock cycles. Processor speed is inversely proportional to clock cycle time.

Ans 3 : Amdahl's Law : defines the theoretical speedup of a system ~~where a~~ when a part of it is improved.

$$\text{Speedup} = \frac{1}{(1-P) + (P/S)}$$

where, P = parallel portion of the program
S = speedup of that portion

It shows that performance improvement is limited by the non-parallel part of the program.

Ans 4 : The datapath consists of hardware components responsible for processing data such as ALU, registers, buses and multiplexers. It is the part of the CPU that actually executes the instructions.

Ans 5 : The control unit directs the operations of the processor by generating control signals. It determines which operation the ALU performs, when data is moved b/w components and which instruction is executed.

Ans 6 : A single cycle processor executes an entire instruction in one clock cycle. It has a simple design but requires a long clock cycle, making it slower and less efficient.

Ans 7 : A multi-cycle processor breaks the instruction execution into multiple steps. Each step is executed in a separate clock cycle. It has a shorter clock cycle ~~st~~ time and better hardware utilization compared to single cycle processor.

Ans 8 : Instruction Pipelining is a technique in which the execution of multiple instructions is overlapped by dividing the instruction cycle into a number of stages. This increases throughput and CPU performance.

Ans 9 : ILP is the ability of a processor to execute multiple instructions simultaneously by exploiting independent instructions in the program.

Ans 10 : Data hazards occur in pipelining when an instruction depends on the result of a previous instruction which has not completed its execution.

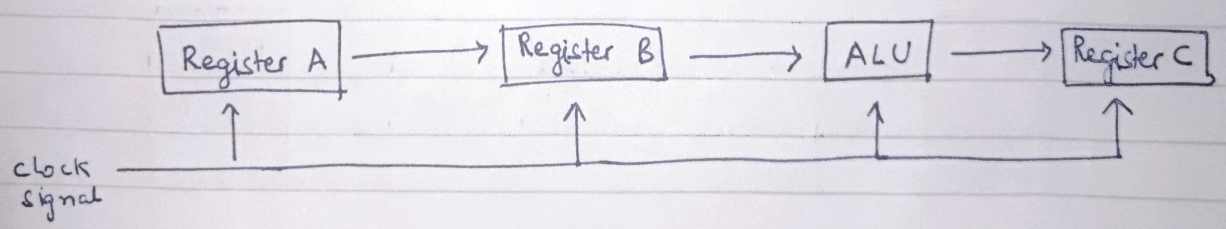
Ans 11 : Control hazards occur due to branch instructions which change the normal flow of the program. The pipeline doesn't know which instruction to fetch next.

Ans 12 : Pipeline hazards can be mitigated using

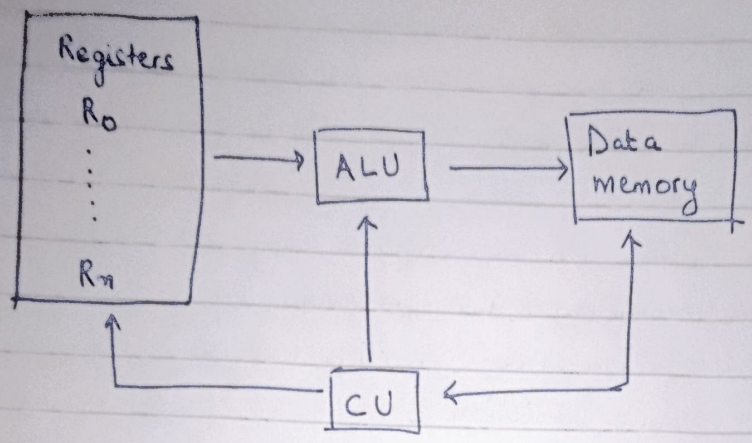
- → Forwarding (data forwarding)
- → Pipeline stalls (inserting bubbles)
- → Branch prediction &
- → Instruction re-ordering (compiler level)

Section – B

Ans 1 :

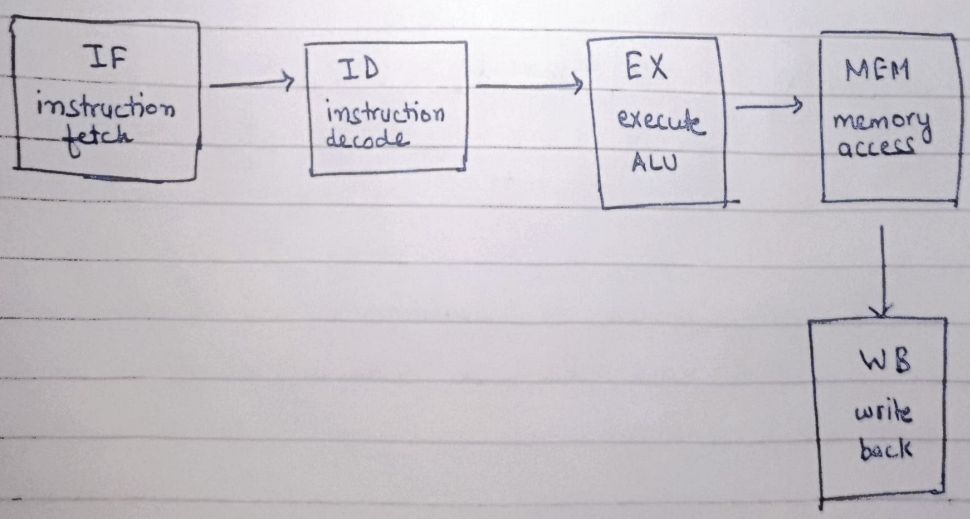


The clock signal synchronizes the operations of all registers and functional units. On each clock pulse, data is transferred and operations are performed.



Data flows from registers to ALU for processing, then to memory or back to registers. CU generates signals to control the whole operation.

Ans 3:



Ans 4 :

a) Single cycle processor

IF	ID	EX	MEM	WB

entire instruction executed in 1 long clock cycle.

b) Multi cycle processor

IF	ID	EX	MEM	WB

each stage is executed in different clock cycles.

Section - C

Ans 1 : Data hazards will occur. They can be solved using data forwarding (bypassing) or pipeline stalls (inserting bubbles).

Ans 2 : $P = 0.8$

$S = 4$

$$\text{speedup} = \frac{1}{(1-P)+(P/S)} \Rightarrow \frac{1}{0.2 + 0.2} \Rightarrow \frac{0.8}{\cancel{40}\ 5}$$

$$\downarrow$$

$$\frac{1}{5} \text{ or } 0.2$$

Ans 3: Multicycle processor has shorter ~~ta~~ clock cycle time and reuses hardware components for different stages. Hence, it is more efficient and has better hardware utilization.

Ans 4: Control hazards occurs. It can be reduced by using branch prediction techniques and instruction pre-fetching.

Section - D

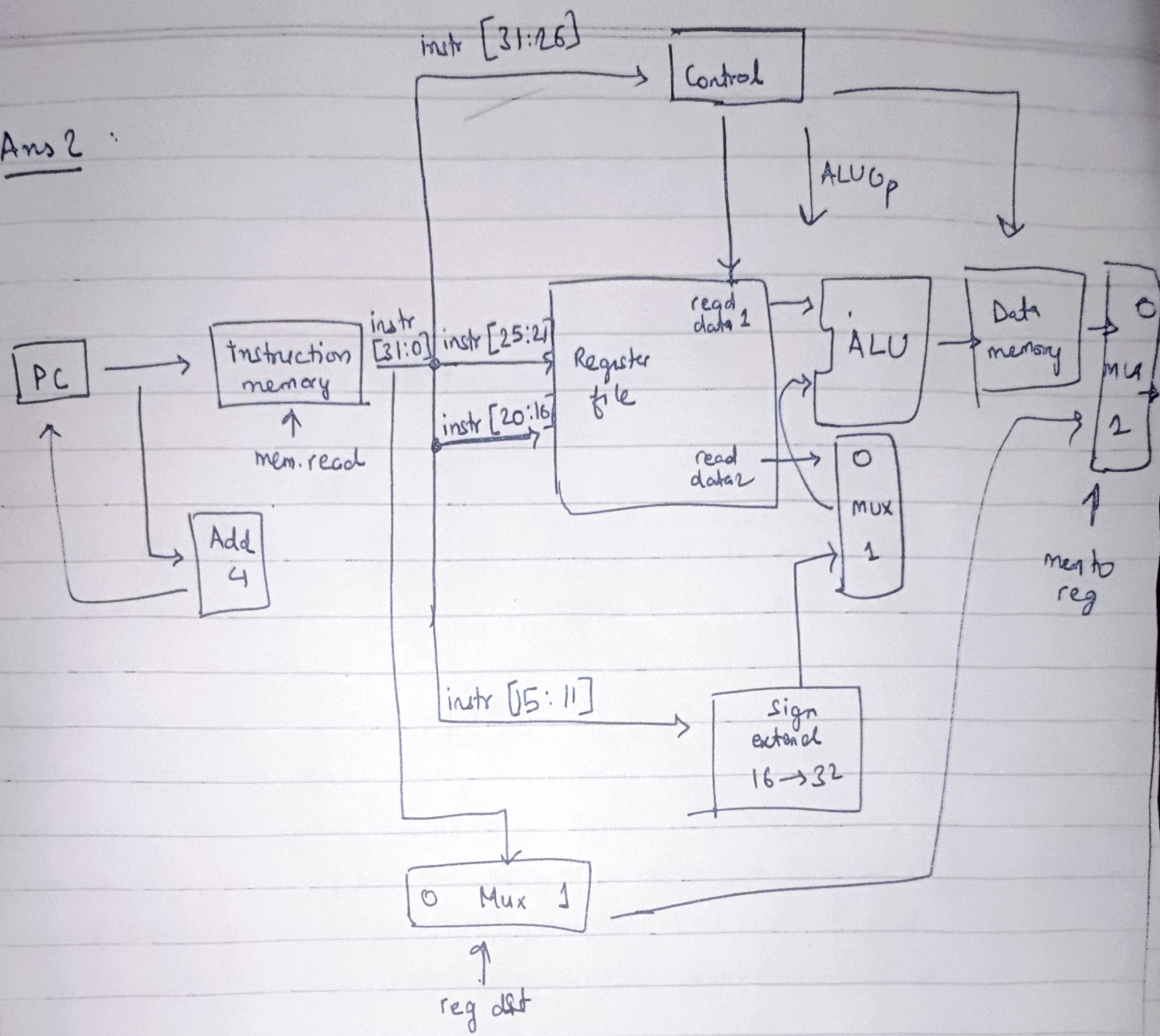
Ans 1:

IF → ID → EX → MEM → WB

EX stage → in this stage, the ALU performs arithmetic or logical operations on the operands. It may also compute branch target addresses. This stage is critical for actual data processing.

Ans 2:

Ans 2 :



use a single memory for both instructions and data (von neumann)

it reduces hardware cost and complexity since we need only one memory interface. The trade-off is that instruction fetches and data accesses cannot happen at the same time

Ans 3 :

Example

ADD R_1, R_2, R_3 ; $R_1 = R_2 + R_3$

SUB R_4, R_1, R_5 ; $R_4 = R_1 - R_5$

prob : SUB needs R_1 before ADD writes it back.

soln : Use data forwarding from EX / MEM stage to EX stage or insert one stall.

Section - E

Ans 1 : Two assumptions while explaining pipelining :

- → it increases instruction throughput by overlapping
- → execution of multiple instructions

~~**Ans 2 :** It will lead to incorrect results or pipeline stalls, reducing performance.~~

~~**Ans 3 :** Multi core processors in smartphones / computers run multiple apps simultaneously.~~

Ans 2 : Pipeline hazards may cause stalls, reducing the performance improvement

Ans 3 : Pipelining is the most challenging because it involves multiple stages, hazards and techniques to handle them, making it complex to visualize and implement.